CommitCV: Project Blueprint

Your commits are your CV, and every claim comes with proof. Final hackathon blueprint.

1. Title and Tagline

Project name: CommitCV

Tagline: Your commits are your CV. Every claim comes with proof.

One-line pitch: Students paste a GitHub URL and get strengths, best-fit roles, skill gaps and a ready resume. Recruiters paste a job description plus GitHub URLs and get a ranked shortlist with proof and interview questions.

Our edge over GitRoll, CodersRank and others: CommitCV is the only analyzer that never states anything it cannot prove. A RAG layer lets the AI speak only from retrieved code, so every claim has a citation or is dropped.

What CommitCV Does and How It Is Different

CommitCV reads a candidate's GitHub, retrieves real evidence from their code, and produces a resume, role matches and a recruiter shortlist where every claim cites its source.

Area	Typical GitHub analyzers	CommitCV
Claims	Scores and AI text, no proof	Every claim cites a file or commit, or it is dropped
Missing data	Low score for quiet profiles	Low-confidence badge and "No evidence found", no penalty
Skills	"Knows Docker"	Verified, Partial or Unsupported label with sources
Recruiter workflow	Static score	Ranked shortlist plus Ask the Repo chat
Student guidance	A score and a graph	Skill gaps, next project, re-analyze to see improvement

Area	Typical GitHub analyzers	CommitCV
Resume	Missing or generic	Bullets built from real repos

One-sentence pitch: CommitCV is hiring on proof, not claims.

2. Problem Statement

- Students have projects on GitHub but do not know how to present them in a resume or which roles to target.
- Recruiters cannot read dozens of GitHub profiles per opening, so they skip them or judge by stars and commit counts.
- Existing tools give vague scores with no explanation, so nobody trusts them.

3. Solution Overview

1. User enters a GitHub URL.
2. The backend fetches profile data through the GitHub API.
3. An LLM pipeline extracts structured, evidence-linked insights.
4. Students see a profile report plus a resume. Recruiters see a match score and a ranked shortlist.

Every output passes through the RAG evidence layer (Section 8), so nothing reaches the screen without a source.

4. Main Dashboard (Landing Page, before login)

Goal: judges must understand the product in 30 seconds.

Sections, top to bottom:

- **Navbar:** logo, How it works, For Students, For Recruiters, Login, Sign up.
- **Hero:** headline "Your commits are your CV", a short subtext, and one big GitHub URL input with an **Analyze** button. No signup needed for the first result.
- **Try a sample:** 3 clickable example profiles so judges see output instantly.
- **Two role cards:** "I am a Student" and "I am a Recruiter", each leading to its own flow.
- **How it works:** 3 steps with icons (Paste URL, AI analyzes repos, Get insights).

- **Feature highlights:** Evidence-linked claims, Skill gap roadmap, One-click resume, Ranked shortlist, Interview questions.
- **Trust strip:** "Advisory scores only. We never use name, photo or location."
- **Footer:** about, privacy, GitHub repo link, team names.

5. Login and Sign Up Page

Keep it simple. Nothing here should take more than 5 minutes to build.

- **Options:** Continue with GitHub (best, since it also gives API access and avoids rate limits) and Email plus password as a fallback.
- **Role selection at sign up:** Student or Recruiter. This decides which dashboard opens.
- **Fields:** name, email, password, role. Recruiters can add a company name.
- **Extras:** forgot password link, clear error messages, and a **Continue as guest (demo)** button so judges never get stuck.
- **After login:** redirect to the Student Dashboard or Recruiter Dashboard by role.

6. Student Dashboard

Goal: give the student something useful and actionable within one screen.

Layout:

- **Top bar:** logo, profile avatar, Re-analyze, Logout.
- **Left sidebar:** Overview, Strengths, Role Match, Skill Gaps, Resume Builder, History.

Overview page:

- **Profile summary card:** avatar, username, bio, public repos, followers, top languages.
- **Overall profile score:** with a **confidence badge** (High, Medium, Low data). Low data shows "results are limited" rather than a harsh score.
- **Score breakdown chart** (radar or bars): Code quality, Project depth, Consistency, Collaboration, Documentation. Each has a one-line reason.

Strengths page:

- A list such as "Strong in Python: built X with tests and CI", each with a **link to the repo or file** as evidence.
- Language and tech stack chart.
- Top 3 standout projects with a short AI summary.

Skill labels: each skill is marked Verified (found in code, tests or config), Partial (mentioned only in a README or commit message) or Unsupported (no evidence found).

Role Match page:

- Best-fit roles (Backend, Frontend, Data, ML, DevOps) with a match percentage and reason.
- Click a role to see what is covered and what is missing.

Skill Gaps page:

- Missing skills for the chosen target role.
- **Next project suggestions**, for example "Build a REST API with auth and a database to qualify for backend roles".
- Optional learning resources.

Growth loop: build the suggested project, click Re-analyze, and see skill labels and scores change in a before/after view.

Resume Builder page:

- Auto-generated sections: summary, skills, projects (bullets taken from real repos), links.
- Edit in place, pick a template, and **Download PDF**.
- Option to tailor the resume to a pasted job description.

History: past analyses so students can see improvement over time.

7. Recruiter Dashboard

Goal: show a decision, not raw data.

Layout:

- **Top bar:** logo, company name, New Job, Logout.
- **Left sidebar:** My Jobs, Candidates, Compare, Saved Shortlist.

Create Job page:

- Job title and a large text box for the **job description**.
- Add candidates by pasting **multiple GitHub URLs** (one per line) or uploading a CSV.
- **Analyze Candidates** button with a step-by-step progress indicator.

- The AI extracts required skills from the description and shows them as editable chips, so the recruiter can correct them.

Ranked Candidates page:

Rank	Candidate	Match %	Matched skills	Missing skills	One-line reason

- Matched skills shown as green chips, missing as red chips.
- Filters and sorting by match, experience signals and language.
- Low-data candidates are flagged with a confidence badge, not rejected.

Candidate Detail page:

- Evidence-linked strengths mapped to job requirements.
- Top relevant repos with summaries.
- **Generated interview questions** based on the candidate's actual code (for example about a design choice in their project).
- Buttons: Shortlist, Reject, Export report.

Ask the Repo (chat): the recruiter types a question such as "Does this candidate write tests?" or "Where is async code used?" and gets an answer with file links, or "No evidence found".

Compare page: pick 2 to 3 candidates and view them side by side.

8. Backend and AI Pipeline

Suggested stack: React or Next.js with Tailwind and shadcn/ui, Node or FastAPI backend, PostgreSQL or Supabase, GitHub REST/GraphQL API with a token, an LLM API for analysis.

Pipeline:

1. **Fetch:** profile, top repos by stars and recent activity, languages, README text, file tree, commit and PR stats, presence of tests and CI config.
2. **Cache:** store results so repeat demos are instant and rate limits are safe.
3. **Analyze:** send compact repo summaries to the LLM and ask for **structured JSON** (strengths, roles, gaps, evidence links, confidence).
4. **Verify:** check that every evidence link actually exists in the fetched data, and drop claims that do not.

5. **Match (recruiter):** extract skills from the job description, compare with candidate evidence, produce a score plus reasons.
6. **Generate:** resume bullets and interview questions.

RAG layer: the boundary that keeps CommitCV honest

Rule: the model may only state what it can retrieve and cite.

1. **Index:** chunk each candidate's code files, READMEs, commit messages and PR descriptions, embed them, and store them in a vector DB (pgvector or Chroma), one namespace per candidate.
2. **Retrieve:** every claim or question first pulls the top matching chunks.
3. **Generate with citations:** the LLM answers only from those chunks and cites the file or commit.
4. **Verify:** any claim without a valid citation is dropped automatically.
5. **Abstain:** if no chunk passes the similarity threshold, the answer is "No evidence found".

Boundary limits: public repos only; name, photo, gender and location never used; off-topic questions refused; no claims about cheating or AI-written code.

Second knowledge base: 8 to 10 role profiles built from real job descriptions, used for role matching and skill gaps.

9. Fairness and Trust (include as one slide)

- Scores are advisory, never an automatic reject.
- No use of name, photo, gender or location in scoring.
- Missing GitHub activity is not treated as a weakness. It lowers confidence instead.
- Every score has a visible "How this score works" panel.

10. 24-Hour Build Plan

Hours	Task
0 to 4	GitHub API fetch and caching, project setup

Hours	Task
4 to 10	RAG indexing, retrieval with citations, abstain rule, Ask the Repo
10 to 16	Landing page, login, student dashboard, resume export
16 to 20	Recruiter dashboard with ranked table and interview questions
20 to 24	Polish, error handling, sample profiles, backup demo video, pitch

If time runs out, keep: RAG with citations, Ask the Repo, skill labels, score breakdown, resume PDF, ranked recruiter table. **Cut first:** dark mode, compare view, shareable card.

11. Demo Script (about 3 minutes)

1. **Problem (20s):** students cannot present GitHub work and recruiters cannot read it at scale.
2. **Student demo (60s):** paste a URL, show the score breakdown, evidence links, skill gaps and the generated resume PDF.
3. **Recruiter demo (60s):** paste a job description and 3 URLs, show the ranked table and an interview question from real code.
4. **Trust (20s):** show the fairness panel and confidence badge.
5. **Close (20s):** vision and next steps.

Tip: analyze a judge's own public profile live, with their permission.

Live moment: a judge types a question into Ask the Repo, then asks something the code cannot prove, and CommitCV answers "No evidence found".

12. Likely Judge Questions

- **How is this different from GitRoll or CodersRank?** CommitCV is built on proof: a RAG layer means nothing is stated without a citation, and Ask the Repo lets recruiters question the code directly.
- **What about students with little GitHub activity?** We show a low-confidence badge and do not penalize them. Resume upload in Phase 3 covers this.

- **How do you avoid AI hallucination?** RAG with citations: the model sees only retrieved code, uncited claims are dropped, and with no evidence it answers "No evidence found".
- **Bias concerns?** Advisory scores, no personal attributes used, transparent reasoning.
- **How accurate is it?** Quote a small test, such as "tested on 30 profiles, X% of skill claims matched repo contents".

13. Roadmap

- **Phase 1 (hackathon MVP):** RAG-grounded GitHub analysis, skill labels, Ask the Repo, student report, resume PDF, recruiter ranking.
- **Phase 2:** growth loop with before/after view, interview questions, compare view, shareable profile card, history tracking.
- **Phase 3:** Resume Truth Check (upload a resume and check each bullet against GitHub evidence), LinkedIn and LeetCode integration, ATS integrations, job board listings.